

CS3160 Lab 6

Building a simple pipeline

What you will do today

The single-cycle simulator executes one instruction before it begins the next one, but a pipelined processor starts executing the next instruction before the previous one has finished. Therefore, five instructions (in case of a 5-stage pipeline) are active in the machine at once. In this lab, you will build a simple pipelined processor.

The individual stages do not change, and so the functions that implement the 5 stages — fetch, decode, operand-fetch, execute, memory, and writeback are called again here. You do not have to edit any of them either because each and every instruction will have to do all these actions anyway as part of its execution. The only change which is needed is *when* each one of these function runs, and how a value is carried from one to the next.

1 One loop iteration is one clock cycle

The single-cycle model's loop ran one cycle per iteration and one *cycle executed one instruction*. The pipeline's loop also runs one cycle per iteration, but in that cycle, the five stages operate on five different instructions.

There is a pipeline register (a latch) that sits between each pair of stages, which holds everything known so far about the instruction (its execution state in some sense). The four pipeline registers are:

IF/ID	the bits that were fetched, and the PC they came from
ID/EX	the decoded instruction and the values of its operands
EX/MEM	what the ALU produced
MEM/WB	what the memory access returned

All these four latches are of the same type, `struct Latch`. This is a simplification (in our simulator). Therefore, a latch is “everything known so far” about an instruction, and later stages know more about an instruction because it been processed more.

A `valid` field is used to indicate whether a stage is working on an instruction or if it is empty/idle. An empty/idle stage happens on the first four cycles, on every stall, and after every flush. An empty latch is called a *bubble*, and `latch_bubble()` creates one.

Read the current latch, write the next latch

Each stage reads the latch in front of it — the state at the *start* of the cycle — and writes a *next* latch. At the end of the cycle (or end of the loop), all four next latches are copied over

the current ones at once. This code-structure allows the stages to be written in any order, but we will write them backwards. This ensures that we do not overwrite any latch with inputs for the subsequent cycle.

The register file is written by WB in the first half of the cycle and read by ID in the second. So an instruction in ID sees what the instruction in WB is writing. This is the convention the textbook uses, and makes a back-to-back dependency cost two cycles instead of three.

2 Hazards

Data Hazards

In this week, you do not have to handle data hazards (this will be done in lab07). The five test programs are written so that no instruction depends on anything computed in the previous two instructions.

Control Hazards

This model decides a branch in MEM, and the machine fetches from the wrong-path in those three cycles. So a taken branch has three instructions behind it that should be flushed. Two functions you write in `src/hazard.cpp` are what the flush is expressed in terms of:

- `redirects_pc(Latch)` — does this latch send the PC somewhere other than straight on? A function of the same name was written in lab05, but that answered the question about an instruction, and this version answers it about a latch. **An empty latch redirects nothing**; a bubble must not be mistaken for the instruction that was squashed out of it.
- `wrong_path_instructions(stage)` — how many were fetched before the redirect took effect. One per stage between fetch and resolution, **inclusive**.

► *Count only the latches that actually held an instruction. Squashing a bubble discards nothing, and counting it gives a **flushed** larger than the number of instructions the program ever fetched — which is the kind of number that looks plausible until you add it up.*

3 The cycle count

With an ideal memory, a five-stage pipeline retires one instruction per cycle once it is full, and it takes four cycles to fill:

$$\text{cycles} = \text{instructions} + 4 + \text{stall_cycles} + \text{flushed_instructions}$$

This week `stall_cycles` is always zero — there is nothing yet that stalls — so the identity is `instructions + 4 + flushed`. Check it by hand on `p1-arith.s`, which has no branches at all and should come out at exactly `instructions + 4`.

That identity is worth more than any single counter. If it does not balance, one of the four numbers is being counted in the wrong place, and that is a much smaller thing to look for than a wrong trace three hundred instructions in.

4 What to write

All of it is in `src/processor_pipelined.cpp`, plus two small functions in `src/hazard.cpp`. Each is marked `TODO(lab 6)` and the comment above it says what it has to do.

WB	retire the instruction, and notice the store that ends the program
MEM	the memory access
EX	the ALU, and where control goes next
ID	decode, ask about hazards, read the registers
IF	fetch — unless the pipeline is held or the program is over
control	redirect the PC and throw away the wrong path
commit	copy the four next latches, and record stats, go to the next cycle

Write the functions in this order. The program tier may stay at zero until all of the above is in place, and then all pass at once. So, work with unit tests which works as each is implemented.

► *An unfinished loop does not hang. If the pipeline runs 256 cycles without retiring anything, the simulator says so and stops — which almost always means IF/ID is never filled, or the next latches are never copied.*

5 Test programs (p2-spaced)

Open `programs/asms/p2-spaced.s`. It is a chain of `addi`, each depending on the one before it, with two `nops` between every pair.

An instruction should not use a value until the instruction producing it has written it back to the register file. This week's pipeline has no way to notice a dependency that is closer, and would quietly use a stale register. Ideally, the compiler would insert these `nops` if it has nothing useful to put in the two intervening instruction slots.

6 Checking your work

```
make          # build build/sim
make test     # build, then unit tests, then programs
make unit     # just the unit tests
```

The unit tests run your pipeline over programs of four or five instructions, so a failure is something you can work through on paper. `docs/` has two notes worth opening before you are stuck: `running-your-simulator.md` for the options, and `debugging-the-pipeline.md` for reading the per-cycle dump — which shows what is in each latch on each cycle and is by far the fastest way to see a pipeline that is not moving.

7 What to hand in

Your folder is collected at the end of the session. Ensure you have saved `details.txt` (name, roll number, machine number), `src/processor_pipelined.cpp` and `src/hazard.cpp`.